

Boas-vindas a Tópicos em Algoritmos e Estruturas de Dados

Bacharelado em Engenharia de Software

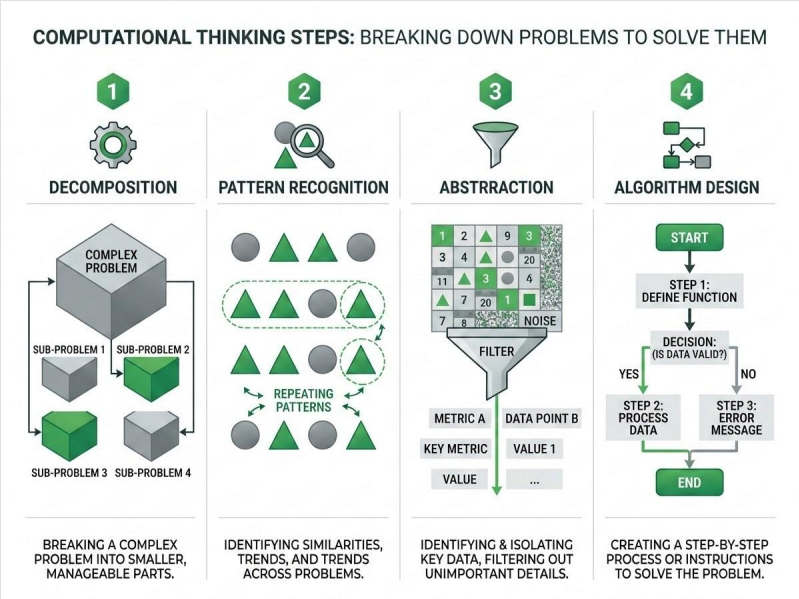
Prof. Dr. Rodrigo Elias Bianchi
Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP) - Campus São Carlos

O que estudaremos nesta disciplina?

Esta disciplina tem como objetivo central aprofundar o desenvolvimento de habilidades avançadas de resolução de problemas computacionais. Focaremos na capacidade de identificar desafios complexos que exigem soluções algorítmicas otimizadas, indo além dos conceitos básicos de programação.

Exploraremos detalhadamente a análise de complexidade, permitindo avaliar o desempenho de diferentes abordagens. O intuito é formar profissionais capazes de projetar sistemas de software de alta qualidade, garantindo que as soluções sejam não apenas corretas, mas também eficientes e escaláveis para o mercado de tecnologia.

Pensamento Computacional e Resolução de Problemas Não Triviais



O pensamento computacional é o alicerce para a criação de soluções em engenharia de software.

Ele envolve a decomposição de problemas complexos em partes menores e gerenciáveis, o reconhecimento de padrões em dados e processos, e a abstração de detalhes desnecessários.

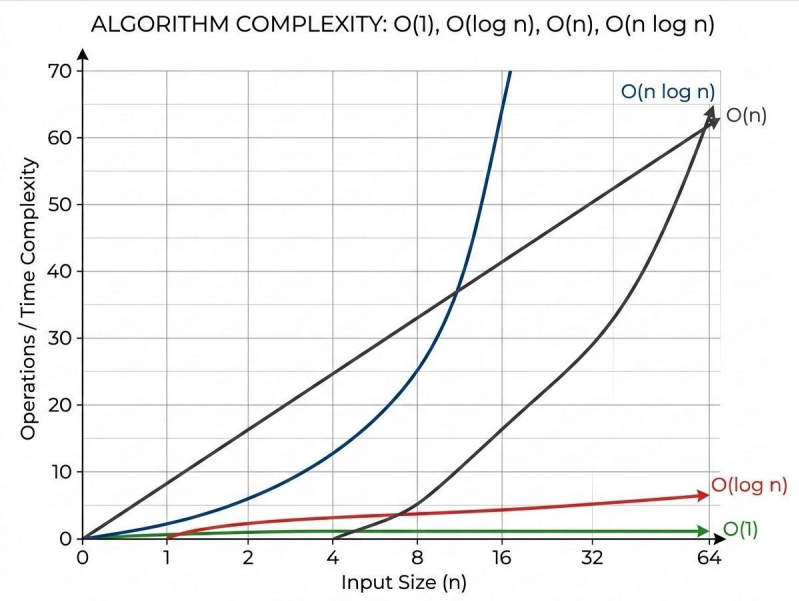
Nesta disciplina, aplicaremos esses pilares para modelar problemas não triviais caracterizados por relações intrincadas entre diferentes domínios de conhecimento. A formulação algorítmica rigorosa nos permitirá traduzir desafios abstratos do mundo real em instruções lógicas precisas e executáveis pelas máquinas.

O Caminho do Aprendizado: Conteúdo Programático

Nosso percurso acadêmico está estruturado em sete blocos fundamentais para o domínio algorítmico:

- **Bloco 1:** Fundamentos de Análise de Complexidade de Tempo e Espaço
- **Bloco 2:** Paradigmas de Projeto (Divisão e Conquista)
- **Bloco 3:** Algoritmos Gulosos (Greedy Algorithms)
- **Bloco 4:** Programação Dinâmica e Otimização
- **Bloco 5:** Estruturas de Dados Avançadas e Árvores Balanceadas
- **Bloco 6:** Teoria dos Grafos e Aplicações Práticas
- **Bloco 7:** Introdução à Programação Competitiva e Heurísticas

Análise de Complexidade: Do Caso Médio ao Pior Caso



A **análise de complexidade** é essencial para prever o comportamento de um algoritmo à medida que o volume de dados cresce.

Estudaremos a **notação Big-O (O)** para descrever o limite superior de tempo e espaço exigido por uma solução.

Avaliaremos o comportamento dos algoritmos em três cenários distintos: o **melhor caso** (condições ideais), o **caso médio** (comportamento esperado no dia a dia) e o **pior caso** (garantia máxima de consumo de recursos).

Também discutiremos a fronteira teórica entre **problemas tratáveis (classe P)** e **intratáveis (classe NP)**.

Paradigmas de Projeto de Algoritmos

Impacto nas Soluções

Os paradigmas de projeto são estratégias padronizadas para a construção de algoritmos eficientes.

Divisão e Conquista

A abordagem de Divisão e Conquista quebra o problema em subproblemas independentes, resolve-os recursivamente e combina os resultados.

Programação Dinâmica

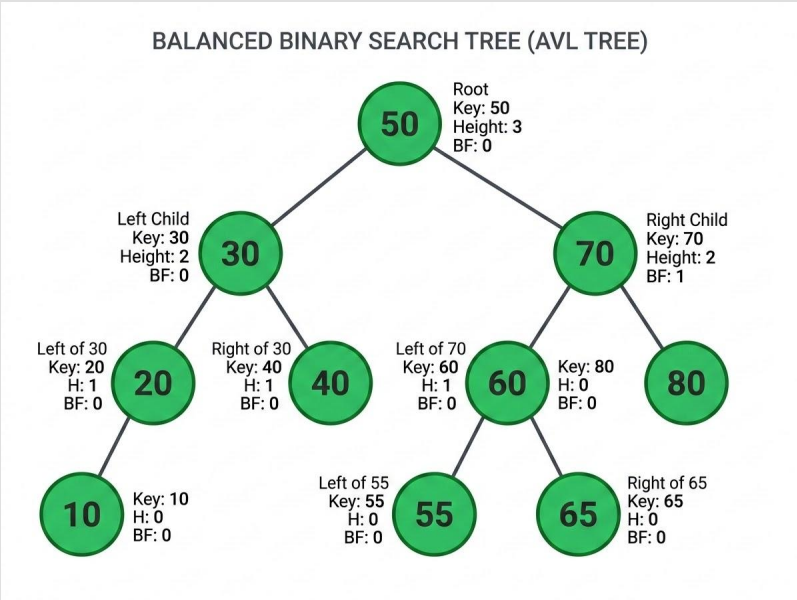
A Programação Dinâmica otimiza soluções ao armazenar resultados de subproblemas sobrepostos, evitando recálculos redundantes e reduzindo drasticamente o tempo de execução.

Algoritmos Gulosos

Já os Algoritmos Gulosos (Greedy) tomam a melhor decisão local a cada passo, buscando um ótimo global.

Importante: A escolha correta do paradigma define o sucesso ou o fracasso de uma implementação.

A Importância da Escolha de Estruturas de Dados Adequadas



Impacto na Viabilidade do Software

As estruturas de dados são a fundação sobre a qual os algoritmos operam. A escolha entre utilizar arrays, listas encadeadas, tabelas hash, pilhas, filas ou árvores balanceadas afeta diretamente a viabilidade de um projeto de software.

Uma estrutura inadequada pode causar gargalos severos de processamento ou estouro de memória. Avaliaremos como o custo das operações de inserção, busca e remoção varia drasticamente entre as estruturas, e como o arranjo correto dos dados em memória dita a eficiência global da aplicação.

Introdução à Programação Competitiva e Treinamento de Desafios

A programação competitiva é um excelente campo de provas para as habilidades de um engenheiro de software. Ela exige a resolução de problemas algorítmicos complexos sob estritas restrições de tempo e memória.

Utilizaremos plataformas online de juízes automáticos para treinar a implementação rápida, limpa e livre de bugs. Essa prática desenvolve o raciocínio lógico sob pressão, aprimora a capacidade de debugar mentalmente o código e solidifica o entendimento prático das estruturas de dados e dos limites da linguagem de programação.

Algoritmos e Estruturas de Dados na Engenharia de Software

Escalabilidade Teórica

Algoritmos eficientes são indispensáveis para o processamento de grandes volumes de dados (Big Data), roteamento em redes, renderização gráfica, sistemas de recomendação e inteligência artificial.

O conhecimento profundo dessas estruturas garante que a infraestrutura tecnológica suporte a escalabilidade exigida pelas aplicações modernas.

Desenvolvimento Prático

A teoria dos algoritmos possui conexão direta com o desenvolvimento prático de sistemas de software no mercado.

No Bacharelado em Engenharia de Software, o foco é criar sistemas de alta qualidade de maneira sistemática e controlada.

Preparação de Carreira

Algoritmos em Processos Seletivos de Big Techs

O domínio de algoritmos e estruturas de dados é o principal critério de avaliação nas entrevistas técnicas das maiores empresas de tecnologia do mundo (Big Techs).

Os processos seletivos exigem que o candidato demonstre, muitas vezes em um quadro branco ou editor colaborativo, a capacidade de formular soluções otimizadas, justificar a complexidade de tempo e espaço (Big-O) e traduzir a lógica em código funcional. O treinamento rigoroso nesta disciplina é o primeiro passo para a aprovação nessas rigorosas avaliações de engenharia.

Importância Estratégica

Preparar soluções otimizadas diretamente no quadro-branco sob as estritas restrições de complexidade de tempo e espaço (Big-O) é o primeiro passo crucial para a aprovação nas rigorosas avaliações das Big Techs.

Bibliografia Essencial

Para o acompanhamento da disciplina é sugerida a consulta à literatura clássica da ciência da computação. Nossa base teórica será fundamentada nas seguintes obras:

Referências Básicas

- CORMEN, T. H. et al.
  Algoritmos: Teoria e Prática.
- DÜRR, C.; VIE, J.
  Programação Competitiva.

Referências Complementares

- HALIM, S.; HALIM, F.
  Competitive Programming.
- SKIENA, S. S.
  The Algorithm Design Manual.
- ZIVIANI, N.
  Projeto de Algoritmos: Com implementações em Pascal e C.

Primeiros Desafios Práticos e Preparação para a Próxima Aula

Para garantir um bom ritmo de aprendizado, os seguintes passos práticos são sugeridos para esta semana:

Passo 01 01

Realizem a leitura do capítulo introdutório sobre Análise de Complexidade no livro do Cormen.

Passo 02 02

Criem uma conta em uma plataforma de programação competitiva (como Beecrowd ou LeetCode) e familiarizem-se com o ambiente de submissão.

Passo 03 03

Revisem os conceitos de estruturas de dados lineares vistos em semestres anteriores, pois eles serão o ponto de partida para nossa próxima aula prática.

Tarefa Prática do Dia: Desafio de Processo Seletivo

Para consolidar o entendimento real das exigências do mercado, realizem a seguinte atividade prática no laboratório hoje:

ATIVIDADE

01

Pesquisa e Análise

Procurem na internet, utilizando a infraestrutura do laboratório, por uma **questão real de processo seletivo** aplicada por alguma grande empresa de tecnologia (Big Tech).

Estudem detalhadamente a sua resposta e preparem uma explicação clara para **apresentar para a turma** em nossas próximas aulas práticas.

FORMATO

Trabalho em Equipe

A atividade deve ser realizada em **grupos de até 5 pessoas**.

Colaborem na discussão do problema, avaliem a complexidade da solução (tempo e espaço utilizando Big-O) e dividam os pontos-chave para a futura apresentação didática.